

THE AGENT WHO THRIVED ON GOOD VIBES

✓ Learning objectives

Upon completing this lab, you will be able to implement agents driven by a type of intrinsic motivation called 'interactional motivation.' This refers to the drive to engage in sensorimotor interactions that have a positive valence while avoiding those that have a negative valence.

✓ Setup

Define the Agent class

```
class Agent:
    def __init__(self, _valences):
        """ Creating our agent """
        self._valences = _valences
        self._action = None
        self._predicted_outcome = None

    def action(self, _outcome):
        """ tracing the previous cycle """
        if self._action is not None:
            print(f"Action: {self._action}, Prediction: {self._predicted_outcome}, Outcome: {_outcome}, "
                  f"Prediction: {self._predicted_outcome == _outcome}, Valence: {self._valences[self._action][_outcome]}")

        """ Computing the next action to enact """
        # TODO: Implement the agent's decision mechanism
        self._action = 0
        # TODO: Implement the agent's anticipation mechanism
        self._predicted_outcome = 0
        return self._action
```

✓ Environment1 class

```
class Environment1:
    """ In Environment 1, action 0 yields outcome 0, action 1 yields outcome 1 """
    def outcome(self, _action):
        # return int(input("entre 0 1 ou 2"))
        if _action == 0:
            return 0
        else:
            return 1
```

✓ Environment2 class

```
class Environment2:
    """ In Environment 2, action 0 yields outcome 1, action 1 yields outcome 0 """
    def outcome(self, _action):
        if _action == 0:
            return 1
        else:
            return 0
```

✓ Define the valence of interactions

```
valences = [[-1, 1],
             [1, -1]]
```

The valence table specifies the valence of each interaction. An interaction is a tuple (action, outcome):

	outcome 0	outcome 1
action 0	-1	1
action 1	1	-1

✓ Instantiate the agent

```
a = Agent(valences)
```

✓ Instantiate the environment

```
e = Environment1()
```

✓ Test run the simulation

```
outcome = 0
for i in range(10):
    action = a.action(outcome)
    outcome = e.outcome(action)
```

```
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: -1
```

Observe that, on each interaction cycle, the agent is mildly satisfied. On one hand, the agent made correct predictions, on the other hand, it experienced negative valence.

✓ PRELIMINARY EXERCISE

Execute the agent in Environment2. Observe that it obtains a positive valence.

Modify the valence table to give a positive valence when the agent selects action 0 and obtains outcome 0. Observe that this agent obtains a positive valence in Environment1.

✓ ASSIGNMENT

Implement Agent2 that selects actions that, it predicts, will result in an interaction that have a positive valence.

Only when the agent gets bored does it select an action which it predicts to result in an interaction that have a negative valence.

In the trace, you should see that the agent learns to obtain a positive valence during several interaction cycles. When the agent gets bored, it occasionally selects an action that may result in a negative valence.

✓ Create Agent2 by overriding the class Agent

```
class Agent2(Agent):

    def __init__(self, _valences):
        super().__init__(_valences)

        # Mémoire : action -> outcome observé
        self.souvenirs = {}

        # Compteur pour savoir quand l'agent s'ennuie
        self.compteur_ennui = 0

    def action(self, _outcome):

        # Ce n'est pas le premier cycle
        if self._action is not None:

            prediction_correcte = (self._predicted_outcome == _outcome)
            valence_obtenue = self._valences[self._action][_outcome]

            print(
                f"Action: {self._action}, "
                f"Prediction: {self._predicted_outcome}, "
                f"Outcome: {_outcome}, "
                f"Prediction correcte: {prediction_correcte}, "
                f"Valence: {valence_obtenue}"
            )

            # Apprendre l'outcome produit par cette action
            self.souvenirs[self._action] = _outcome

            # Si l'interaction est positive et bien prédite,
            # l'agent commence progressivement à s'ennuyer
            if prediction_correcte and valence_obtenue > 0:
                self.compteur_ennui += 1
            else:
                self.compteur_ennui = 0
```

```
# CHOIX DE LA PROCHAINE ACTION
```

```
# Si l'agent s'ennuie après 4 cycles
```

```
if self.compteur_ennui >= 4:
```

```
    # Il essaye l'autre action
```

```
    self._action = 1 - self._action
```

```
    # On remet le compteur à zéro
```

```
    self.compteur_ennui = 0
```

```
else:
```

```
    # Chercher une action connue de valence positive
```

```
    action_positive = None
```

```
    for action, outcome in self.souvenirs.items():
```

```
        valence = self._valences[action][outcome]
```

```
        if valence > 0:
```

```
            action_positive = action
```

```
            break
```

```
    # Si on connaît une action positive, on la choisit
```

```
    if action_positive is not None:
```

```
        self._action = action_positive
```

```
    # Sinon, essayer une action encore inconnue
```

```
    elif 0 not in self.souvenirs:
```

```
        self._action = 0
```

```
    elif 1 not in self.souvenirs:
```

```
        self._action = 1
```

```
    # Si tout est connu mais négatif, on change d'action
```

```
    else:
```

```
        self._action = 1 - self._action
```

```
# Prédiction de l'outcome de la prochaine action
```

```
self._predicted_outcome = self.souvenirs.get(self._action, 0)
```

```
return self._action
```

✓ Test your Agent2 in Environment1

```
a = Agent2(valences)
e = Environment1()
outcome = 0
for i in range(20):
    action = a.action(outcome)
    outcome = e.outcome(action)
```

```
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: -1
```

✓ Test your Agent2 in Environment2

```
a = Agent2(valences)
e = Environment2()
outcome = 0
for i in range(20):
    action = a.action(outcome)
    outcome = e.outcome(action)
```

```
Action: 0, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
```

```
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
```

- Test your agent with a different valence table

Note that, depending on the valence that you define, it may be impossible for the agent to obtain a positive valence in some environments.

```
# Choose different valences
valences = [[1, -1],
            [-1, 1]]
# Run the agent
a = Agent2(valences)
e = Environment2()
outcome = 0
for i in range(20):
    action = a.action(outcome)
    outcome = e.outcome(action)
```

[illegible]

```
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
```

✓ Report

Explain what you programmed and what results you observed. Export this document as PDF including your code, the traces you obtained, and your explanations below (no more than a few paragraphs):

Implémentation

J'ai implémenté Agent2 en ajoutant une mémoire des outcomes observés pour chaque action ainsi qu'un compteur d'ennui. La mémoire permet à l'agent d'apprendre progressivement quel outcome est produit par chaque action. À partir de cette prédiction, l'agent consulte la table des valences pour déterminer si l'interaction prévue est positive ou négative.

L'agent cherche en priorité une action dont l'interaction prédite possède une valence positive. Lorsqu'il obtient plusieurs interactions positives et correctement prédites, son compteur d'ennui augmente. Après plusieurs cycles, il explore l'autre action. Si aucune action positive n'est connue, l'agent explore également les différentes actions afin d'apprendre leurs outcomes.

Résultats observés

Avec la première table de valences dans Environment1, les deux interactions possibles ont une valence négative : l'action 0 produit l'outcome 0 et l'action 1 produit l'outcome 1, qui ont toutes les deux une valence de -1. L'agent apprend correctement les outcomes, mais il ne peut pas trouver d'interaction positive.

Dans Environment2, avec la même table de valences, l'action 0 produit l'outcome 1 et l'action 1 produit l'outcome 0. Ces deux interactions ont une valence positive. Après une première erreur de prédiction, l'agent apprend les outcomes et obtient ensuite principalement des prédictions correctes avec une valence positive. Il change occasionnellement d'action lorsque le compteur d'ennui est atteint.

Enfin, avec la nouvelle table de valences et Environment2, les deux interactions possibles ont une valence négative. Il est donc impossible pour l'agent d'obtenir une valence positive. L'agent apprend néanmoins correctement les conséquences des deux actions et alterne entre elles puisqu'aucune action positive n'est disponible.

